LayerZero Console

Security Assessment

March 25th, 2026 — Prepared by OtterSec

Nicholas R. Putra nicholas@osec.io

Zhenghang Xiao kiprey@osec.io

Jinwoo Lee jin@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-LCS-ADV-00 | Silent Truncation Risk due to Overflow 6

General Findings **7**

OS-LCS-SUG-00 | Redundant Initializer Invocation Risk 8

OS-LCS-SUG-01 | Prevention of No-Op Admin Updates 9

OS-LCS-SUG-02 | Code Maturity 10

Appendices

Vulnerability Rating Scale **11**

Procedure **12**

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

LayerZero engaged OtterSec to assess the `console` program. This assessment was conducted between January 19th and January 22nd, 2026. Follow-up audits to assess remediation's were conducted between February 25th and February 26th, 2026, and between March 11th and March 15th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 4 findings throughout this audit engagement.

In particular, we identified an overflow vulnerability during conversion from a local-decimal amount into a shared-decimal amount by casting the result to uint64 without any bounds check (OS-LCS-ADV-00).

We also suggested reverting when the new address equals the current one prevents no-op admin updates and potential misconfiguration (OS-LCS-SUG-01). Additionally, we recommended updating the initializer chain to ensure each parent initializer is invoked only once (OS-LCS-SUG-00).

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/LayerZero-Labs/console-audit. This audit was performed against commit cb3b732. The remediation audits were performed against 79cefc3, b1d3489, 47a1dab, and cb3b732.

A brief description of the program is as follows:

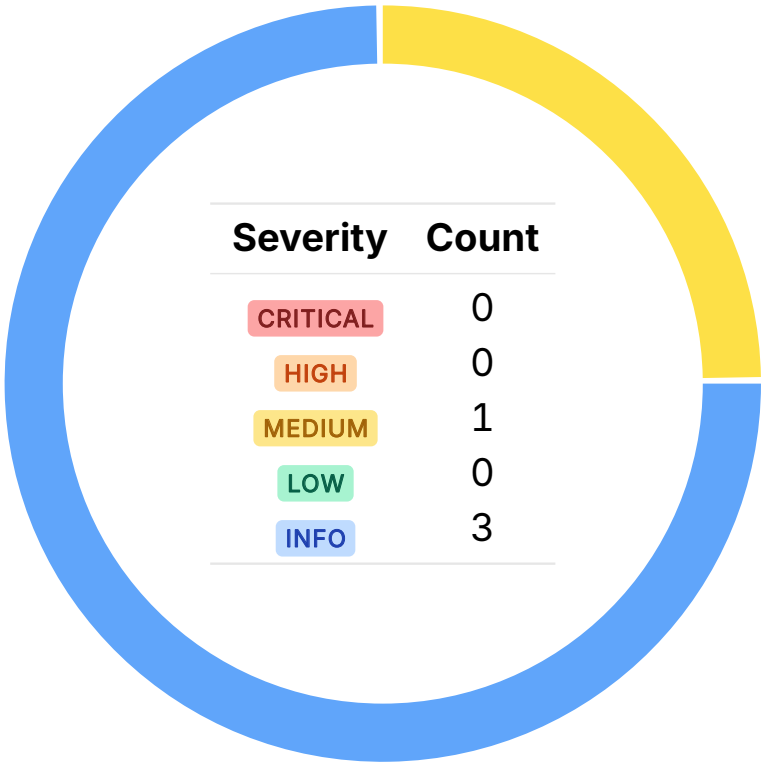
Name	Description
console	It implements a LayerZero-based OFT adapter that enables cross-chain transfers of an existing ERC-20 by custodially locking tokens on the source chain and unlocking them on the destination chain. They add governance-controlled safety and economics layers, including pausability, rate limiting, and native ERC-20 fee collection, without minting or burning supply.

© 2026 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 4 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	1
LOW	0
INFO	3

© 2026 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-LCS-ADV-00	MEDIUM	RESOLVED	`_toSD` casts to `uint64` without an overflow check, resulting in silent truncation if the shared-decimal amount exceeds `uint64` limits.

© 2026 Otter Audits LLC. All Rights Reserved.

Silent Truncation Risk due to Overflow MEDIUM OS-LCS-ADV-00

Description

`DecimalUtils::_toSD` converts a local-decimal amount into a shared-decimal amount by dividing by `DECIMAL_CONVERSION_RATE` and then casting the result to `uint64` without any bounds check. As a result, if `_amountLD / DECIMAL_CONVERSION_RATE` exceeds `type(uint64).max`, `amountSD` will silently overflow and be truncated, rather than reverting with `AmountSDOverflowed`.

>_ contracts/oft-upgradeable/utils/DecimalUtils.sol SOLIDITY

```solidity
/**
 * @notice Converts an amount from local decimals into shared decimals.
 * @param _amountLD Amount in local decimals
 * @return amountSD Amount in shared decimals
 */
function _toSD(uint256 _amountLD) internal view virtual returns (uint64 amountSD){
    return uint64(_amountLD / DECIMAL_CONVERSION_RATE);
}
```

Remediation

Add an explicit overflow guard to prevent the risk of silent truncation.

Patch

Resolved in 79cefc3.

© 2026 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-LCS-SUG-00	Redundant initializer calls in the inheritance chain may result in the same base initializer to execute multiple times.
OS-LCS-SUG-01	Revert when the new address equals the current one prevents no-op admin updates and potential misconfiguration.
OS-LCS-SUG-02	Several areas in the codebase would benefit from additional defensive checks to improve robustness and reduce the risk of unexpected behavior under edge conditions.

© 2026 Otter Audits LLC. All Rights Reserved.

Redundant Initializer Invocation Risk

OS-LCS-SUG-00

Description

Some parent initializers may be reached through multiple inheritance paths (`__OFTCoreRBAC_init` calling `__OAppOptionsType3Base_init` twice), resulting in redundant initializer execution. While currently harmless, this creates upgrade risk if the initializer logic changes in future versions.

Remediation

Update the initializer chain to ensure each parent initializer is invoked only once.

Patch

Resolved in 47a1dab.

© 2026 Otter Audits LLC. All Rights Reserved.

Prevention of No-Op Admin Updates OS-LCS-SUG-01

Description

Revert when `setMsgInspector` or `setGuard` is called with the current address to prevent no-op configuration updates, avoiding unnecessary gas consumption and reducing the risk of inadvertent misconfiguration.

Remediation

Revert in case the new address is same as the current address.

Patch

This was acknowledged by the LayerZero development team.

© 2026 Otter Audits LLC. All Rights Reserved.

Code Maturity

OS-LCS-SUG-02

Description

1. `OFTDecimalUtils` computes `10 ** (LOCAL_DECIMALS - SHARED_DECIMALS)` without enforcing an upper bound on the decimal gap. Consider adding a limit to prevent pathological conversion rates.
2. `RateLimiterBaseUpgradeable` performs unchecked casts from `uint256` to `uint96` for usage values and to `uint40` for timestamps. Consider using safe casting to revert explicitly rather than silently truncating.
3. The allowlist and denylist logic does not enforce mutual exclusivity, allowing an address to exist in both sets simultaneously. Consider ensuring that adding an address to one set automatically removes it from the other to prevent conflicting access control state.

Remediation

Incorporate the above suggestions.

Patch

These were acknowledged by the LayerZero development team.

© 2026 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.